

Metadata of the chapter that will be visualized in SpringerLink

Book Title	Machine Learning for Cyber Security	
Series Title		
Chapter Title	MalScan AI: An AI-Powered Tool for Malicious Source Code Detection	
Copyright Year	2027	
Copyright HolderName	The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd.	
Author	Family Name	Nor
	Particle	
	Given Name	Rizal Mohd
	Prefix	
	Suffix	
	Role	
	Division	Department of Computer Science
	Organization	Kulliyyah of Information and Communication Technology, IIUM
	Address	Kuala Lumpur, Malaysia
	Email	rizalmohdnor@iium.edu.my
	ORCID	http://orcid.org/0000-0002-8994-2234
Author	Family Name	Amiruzzaman
	Particle	
	Given Name	Md.
	Prefix	
	Suffix	
	Role	
	Division	Department of Computer Science
	Organization	West Chester University
	Address	West Chester, PA, USA
	Email	mamiruzzaman@wcupa.edu
	ORCID	http://orcid.org/0000-0002-2292-5798
Corresponding Author	Family Name	Islam
	Particle	
	Given Name	Md. Rajibul
	Prefix	
	Suffix	
	Role	
	Division	Department of Data Science and Engineering
	Organization	Bangladesh University of Business and Technology
	Address	Dhaka, Bangladesh
	Email	md.rajibul.islam@bubt.edu.bd
	ORCID	http://orcid.org/0000-0003-0565-6917
Author	Family Name	Kahar
	Particle	

	Given Name	Muhammad Afnan Hariz Bin Abdul
	Prefix	
	Suffix	
	Role	
	Division	Department of Computer Science
	Organization	Kulliyyah of Information and Communication Technology, IIUM
	Address	Kuala Lumpur, Malaysia
	Email	afnanhariz64@iium.edu.my
Author	Family Name	Zakir
	Particle	
	Given Name	Muhammad Uzair Bin Mohammad
	Prefix	
	Suffix	
	Role	
	Division	Department of Computer Science
	Organization	Kulliyyah of Information and Communication Technology, IIUM
	Address	Kuala Lumpur, Malaysia
	Email	uzairmohdzakir79@iium.edu.my
Abstract	Source code malware poses an increasing security risk in modern software development, particularly for students and novice programmers who may not recognize malicious patterns in code. Existing malware analysis platforms such as VirusTotal and Hybrid Analysis mainly focus on executable files rather than raw source code. This paper presents MalScan AI, a web-based malware detection platform for Python source code. The system analyzes uploaded .py files without execution using static code analysis and machine learning techniques. Static features representing suspicious behaviors are extracted from the source code and used to classify files as benign or malicious. The platform is implemented using Flask, Bootstrap, JavaScript, and Firebase for user authentication and result management. MalScan AI aims to support early-stage malware detection and improve secure coding awareness among learners and developers.	
Keywords (separated by '-')	malware detection - Python source code - static analysis - machine learning - Random Forest - secure coding	



MalScan AI: An AI-Powered Tool for Malicious Source Code Detection

Rizal Mohd Nor¹ , Md. Amiruzzaman² , Md. Rajibul Islam³ ,
Muhammad Afnan Hariz Bin Abdul Kahar¹,
and Muhammad Uzair Bin Mohammad Zakir¹

- ¹ Department of Computer Science, Kulliyyah of Information and Communication Technology, IIUM, Kuala Lumpur, Malaysia
{rizalmohdnor,afnanhariz64,uzairmohdzakir79}@iiu.edu.my
- ² Department of Computer Science, West Chester University, West Chester, PA, USA
mamiruzzaman@wcupa.edu
- ³ Department of Data Science and Engineering, Bangladesh University of Business and Technology, Dhaka, Bangladesh
md.rajabul.islam@bubt.edu.bd

Abstract. Source code malware poses an increasing security risk in modern software development, particularly for students and novice programmers who may not recognize malicious patterns in code. Existing malware analysis platforms such as VirusTotal and Hybrid Analysis mainly focus on executable files rather than raw source code. This paper presents MalScan AI, a web-based malware detection platform for Python source code. The system analyzes uploaded .py files without execution using static code analysis and machine learning techniques. Static features representing suspicious behaviors are extracted from the source code and used to classify files as benign or malicious. The platform is implemented using Flask, Bootstrap, JavaScript, and Firebase for user authentication and result management. MalScan AI aims to support early-stage malware detection and improve secure coding awareness among learners and developers.

[AQ1]

Keywords: malware detection · Python source code · static analysis · machine learning · Random Forest · secure coding

1 Introduction

The software development world is rapidly changing with the evolution of open-source resources and collaborative software. Although such technologies enhance faster software development, they also raise serious concerns regarding software security risks, such as the unintended injection of malicious code into software systems. Fresh developers and students are highly susceptible to such risks because they mainly focus on functionality and may not have enough skills to recognize malware hidden in third-party scripts.

Despite the presence of such risks, there is a great demand for an automated tool in the process of analyzing source code. Current solutions such as VirusTotal mainly concentrate on binary code such as executable files generated from compiled programs [1]. Moreover, the process of analyzing interpreted programming languages such as Python is not optimized for simple source or text-based submissions. This creates the demand for an automated tool capable of finding malicious code in the initial stages of the Software Development Life Cycle.

This study addresses this challenge by proposing MalScan AI, a web-based detection system intended to detect malicious patterns without executing Python source code. The specific objectives are as follows:

- To design and develop an AI-based malware detection system capable of automatically analyzing and classifying malicious patterns in Python source code.
- To apply static code analysis and machine learning techniques to extract meaningful features that represent potentially malicious behaviors.
- To provide automated classification feedback that assists users in identifying suspicious source code segments.
- To evaluate the performance of the proposed system using benign and malicious Python source code samples, with emphasis on detection accuracy and false positive rates.

This work contributes to the field of software security by providing a focused solution for Python source code malware detection. Although the current system is limited to static analysis of Python files and relies partly on synthetic data to enhance the training dataset, it provides a practical foundation for early-stage malware detection in software development.

1.1 Contributions

The primary contributions of this paper are summarized as follows:

1. A web-based platform for detecting malicious Python source code without executing uploaded files.
2. A hybrid feature extraction framework that combines AST-based static analysis with heuristic behavioral indicators to identify potentially malicious code patterns.
3. An explainable reporting mechanism that highlights suspicious code segments and provides downloadable scan reports for users.
4. An empirical evaluation of multiple machine learning classifiers for malicious Python source code detection, demonstrating the effectiveness of Random Forest for the collected dataset.
5. A practical educational tool that promotes secure coding awareness among students, novice developers, and cybersecurity learners.

2 Literature Review

In order to understand the existing state of malicious source code detection and pinpoint essential knowledge gaps, a review of existing tools was performed. This section examines existing technologies, ranging from conventional malware scanners to source code analyzers, and points out inherent weaknesses when dealing with basic Python source code.

2.1 Existing Detection Systems

Binary and Executable Analysis. VirusTotal and Hybrid Analysis are among the leading tools used in malware detection. VirusTotal combines more than 70 antivirus engines to analyze files and URLs with high detection rates based on collective consensus [2]. Hybrid Analysis relies on dynamic sandbox analysis, where it runs files in a controlled environment and analyzes API calls and registry interactions [3].

Although these tools are useful for reverse-engineering and analyzing binary code, such as executable and document files, they tend to be inefficient when processing raw source code files. In the case study, neither environment is capable of effectively analyzing Python scripts as source code.

Static Code and Web Analysis. In contrast to binary-oriented tools, static source code analysis systems analyze human-readable source code. As a representative tool in the static analysis community, SonarQube supports continuous code quality monitoring. It is incorporated with DevOps processes to analyze software bugs and security flaws in Java, Python, and other programming platforms [4]. Nevertheless, it depends on pre-developed analysis rules that identify inefficient coding practices while not directly focusing on malicious intent. As such, it is challenging for such tools to identify obfuscated malware.

Malware Scan focuses on web-based source code analysis, particularly PHP and JavaScript malware commonly found in WordPress-related exploits [5]. Although it can process raw source code, its scope is limited to web technologies and does not cover general-purpose Python malware detection or behavioral machine learning analysis. This highlights a gap in existing tools for detecting malicious patterns in Python source code.

Comparative Analysis. The limitations of the systems above indicate that the security ecosystem lacks a dedicated tool to analyze raw Python source code using advanced behavioral or machine learning techniques. Table 1 compares the functional capabilities of existing tools with the proposed system.

MalScan AI is intended to overcome the shortfalls identified in Table 1. By adapting the multi-engine validation approach of VirusTotal and the behavioral inference objective of Hybrid Analysis, MalScan AI fills the gap between static analysis and malicious source code detection.

Table 1. Feature Comparison of Related Tools

Feature/Tool	VirusTotal	Hybrid Analysis	SonarQube	Malware Scan	MalScan AI
Supports raw source code (e.g., .py)	No	No	Yes	Yes	Yes
Static code analysis	No	No	Yes	Yes	Yes
Detects security vulnerabilities	No	No	Yes	Yes	Yes
Behavioral/malware detection	Yes	Yes	No	No	Yes
Suitable for malware source code review	No	No	No	No	Yes

MalScan AI differs from SonarQube, which is focused on quality, and VirusTotal, which is focused on binaries, because it is directly aimed at raw Python source code. MalScan AI goes beyond simple signature matching by considering machine learning methods, including LSTM networks and Transformer models in related work [6], to identify abnormalities or malicious activity directly through source code.

2.2 Academic Review

Several related papers were reviewed to explore the effectiveness of machine learning techniques for static malware detection. Although the commercial tools highlighted earlier primarily include dynamic sandboxing functionality, a number of papers have covered machine learning techniques in static malware detection. These papers support the proposed approach of using feature extraction and classification as a viable alternative to dynamic sandboxing.

Static Analysis in Executable Files. Yuk and Seo proposed a malware detection method using static analysis of Portable Executable (PE) header attribute values. These attributes, combined with ensemble machine learning algorithms, particularly stacking algorithms, improved detection accuracy to 95.2%, outperforming standalone algorithms [7]. This proves that static attributes, when combined with ensemble algorithms, can provide high detection accuracy without executing malware.

This approach verifies the fundamental idea of static analysis, supporting MalScan AI in following a method involving the extraction of non-execution features from source code and abstract syntax structures rather than binary formats.

Machine Learning Model: Random Forest Classifier. Sari and Aci [8] assessed diverse machine learning algorithms for static malware analysis, focusing on K-Nearest Neighbors, Naive Bayes, Decision Trees, and Random Forest classification algorithms. The study concluded that the best result was obtained by the Random Forest algorithm with a classification accuracy of 97.75%, while Naive Bayes obtained the lowest accuracy at 53%. This endorses Random Forest as an effective algorithm for malicious pattern identification in static malware analysis.

Based on these findings, there are empirical justifications for using Random Forest as the primary classifier for MalScan AI. This helps MalScan AI classify accurately while being resistant to overfitting.

Use of Synthetic Dataset for Training. To mitigate data scarcity and class imbalance in malware detection datasets, Matsobane and Mokwena explored a flow-based generative model for producing realistic synthetic malware samples. The experiment used the CICMalDroid2020 dataset as a baseline with 11,598 samples. Through extracted patterns, the generative model produced 13,402 samples. The Random Forest classifier obtained a precision rate of 93% and a recall rate of 100% when trained on this model [9]. This indicates that synthetic data can help address imbalanced datasets while improving predictive performance for detecting malware.

This supports the strategic approach of MalScan AI in using synthetic examples of malicious Python code during training to alleviate limitations from data scarcity and class imbalance.

Malware Detection Approaches. Aslan and Samet investigated malware detection methodologies including signature-based, behavior-based, and heuristic approaches, analyzing feature extraction techniques such as n-grams, Control Flow Graphs (CFG), and API calls [10]. The review highlights that signature-based methods are computationally efficient but fail against obfuscated and unknown threats, while heuristic and machine learning-based approaches offer improved detection rates for polymorphic malware despite possible increases in false positives.

These findings support MalScan AI's hybrid use of static structural features and machine learning classifiers to detect malicious behavior without the overhead of dynamic analysis.

Malicious Code Detection Using Machine Learning. Kumar proposed a framework for classifying malware into nine distinct classes using static features, specifically file size, hex-code uni-grams, and hex-code bi-grams. Multiple classifiers were trained, and the model with the highest detection accuracy was selected to minimize false positives using confusion matrices for evaluation [11]. The study indicates that ensemble methods such as Random Forest and Gradient Boosting generally elevate detection accuracy.

These findings validate the effectiveness of ensemble learning classifiers, particularly Random Forest, for static malware detection, although MalScan AI applies these models to source-code-level features rather than binary hex-code representations.

Table 2. Summary of Reviewed Papers

Paper Title	Findings	Adoption/Relevance to MalScan AI
Static Analysis and Machine Learning-based Malware Detection System using PE Header Feature Values	Showed the ability to attain accuracy rates of 95.2% using ensemble learning on static PE header features without executing the file.	Validates the basic idea of using static analysis in MalScan AI through feature extraction without execution.
Detection of Malware by Static Analysis Using Machine Learning Methods	Tested different algorithms and showed that Random Forest obtained the highest accuracy of 97.75%.	Justifies the selection of Random Forest as the primary classifier for MalScan AI.
The Impact of Synthetic Data Augmentation on Malware Detection Using a Random Forest Classifier	Proved that synthetic data generation for handling class imbalance improves precision and recall using Random Forest.	Supports the consideration of synthesized malicious samples to mitigate small class distributions during training.
Malware Detection Approaches	Heuristic and machine learning-based methods outperform signature-based techniques in detecting obfuscated and polymorphic malware, though with higher false-positive risk.	Supports MalScan AI's hybrid use of static structural features and machine learning classifiers without relying on dynamic analysis.
Malicious Code Detection Using Machine Learning	Ensemble classifiers, particularly Random Forest, demonstrated superior detection performance in multi-class malware classification using static features.	Validates ensemble learning for static malware detection and supports the use of Random Forest in MalScan AI.
Machine Learning Techniques for Python Source Code Vulnerability Detection	The BiLSTM model achieved an average accuracy of 98.6% and an F1-score of 94.7%, outperforming traditional classifiers.	Supports source-code-level static analysis while suggesting future deep learning extensions for MalScan AI.



Fig. 1. System workflow of MalScan AI.

Machine Learning Techniques for Python Source Code Vulnerability Detection. Farasat and Posegga investigated the detection of Python-specific vulnerabilities, such as SQL injection and XSS, by comparing a deep learning BiLSTM model against traditional classifiers using Word2Vec embeddings for source code token representation. Experimental results showed that the BiLSTM model achieved an average accuracy of 98.6% and an F-score of 94.7%, significantly outperforming traditional machine learning models across seven vulnerability categories [12].

These findings contrast with MalScan AI's primary reliance on traditional classifiers and suggest that integrating sequential deep learning could improve detection of complex Python source code patterns.

Summary of Reviewed Papers. Table 2 summarizes the reviewed papers and their relevance to MalScan AI.

3 Methodology

MalScan AI is designed as a static-analysis-based malware detection system that identifies malicious behavior directly from Python source code. The system follows a structured processing pipeline that transforms raw source code into discriminative features, which are then analyzed by a machine learning classifier to determine whether the code is benign or malicious. This section presents an overview of the system workflow and explains the role of each processing stage.

3.1 System Workflow

The workflow of MalScan AI is illustrated in Fig. 1 and consists of five sequential stages.

First, Python source code is provided as the system input. The submitted file is processed without execution, ensuring that no malicious payload is triggered during analysis. Next, static code analysis is performed using Abstract Syntax Tree (AST) parsing to extract the structural representation of the program. This enables the system to analyze code behavior at a semantic level rather than relying solely on raw text patterns.

In the feature extraction stage, both syntactic and heuristic behavioral features are derived from the parsed AST. These features capture indicators of malicious behavior such as suspicious function calls, file system operations, encoding techniques, and network-related activities. The extracted features are then transformed into a numerical representation suitable for machine learning.

Subsequently, the feature vector is passed to a trained machine learning classification model, which evaluates the likelihood of the source code being malicious. Finally, the system outputs the classification result along with a confidence score, allowing users to assess the severity and reliability of the detection.

3.2 Datasets

A supervised learning approach is employed using a dataset of benign and malicious Python source code collected from public GitHub repositories [13–15]. Benign samples consist of legitimate Python programs from various application domains, while malicious samples include real-world malicious code and synthetically generated variants used to address data scarcity and class imbalance.

Table 3 summarizes the dataset composition used in this study. Each source file is labeled as benign (0) or suspicious (1) and processed independently. Stratified sampling is applied to split the dataset into training and testing subsets while maintaining class balance. Table 4 provides representative examples of features extracted by the proposed system.

Table 3. Dataset Composition

Category	Samples	Percentage
Benign Python Files	534	50.1%
Malicious Python Files	532	49.9%
Total	1066	100%

Table 4. Representative Features Used for Classification

Feature	Type
eval() usage	Binary
exec() usage	Binary
os.system() calls	Binary
subprocess invocation	Binary
Network API usage	Integer
File deletion operations	Integer
Base64 decoding usage	Binary
String entropy score	Continuous
Thread creation count	Integer
Dynamic imports	Binary

3.3 Synthetic Data Augmentation

Due to the limited availability of publicly labeled malicious Python source code, a subset of the malicious samples was augmented using controlled syntactic transformations. These transformations included variable renaming, formatting modifications, comment insertion or removal, and simple obfuscation techniques. The objective was to increase dataset diversity while preserving the original malicious behavior of the source code.

The synthetic samples were used to supplement the training dataset and improve model robustness against variations in code structure and style.

3.4 Dataset Partitioning

The dataset was partitioned using an 80:20 stratified train-test split at the file level to preserve class balance during model training and evaluation. Each source code file was treated as an independent sample throughout the experimental process.

While stratified sampling was applied to maintain balanced class distributions, explicit malware-family-based partitioning was not performed. Future work will investigate family-aware partitioning strategies to further evaluate model generalization across unseen malware variants.

3.5 Feature Extractor

MalScan AI employs a hybrid feature extraction strategy that integrates static code analysis with heuristic behavioral analysis to transform raw Python source code into structured, machine-readable feature representations. This combined approach enables the system to capture both syntactic characteristics and higher-level behavioral indicators, improving detection performance compared to basic keyword or frequency-based techniques.

Static code analysis is performed by parsing the AST of each source file. Structural features extracted at this stage include control flow constructs, function and class definitions, nesting depth, and code complexity indicators. In addition, lexical features such as import statements, function invocations, string literals, and dynamic execution mechanisms are identified to capture potentially suspicious coding patterns.

Building upon the static analysis foundation, a heuristic behavioral analysis engine evaluates the source code for malware-related behaviors commonly observed in malicious scripts. These behaviors include abnormal file system operations, encoding and obfuscation techniques, network-related activities, and system-level API usage. The extracted heuristic features are organized into multiple behavioral categories, as summarized in Table 5.

All extracted features are represented numerically to form a deterministic feature vector suitable for machine learning [9]. Boolean features are encoded as binary values, count-based features are normalized to scalar representations, and missing values are replaced with zeros to maintain matrix consistency. In

addition, a weighted scoring mechanism may be applied to emphasize high-risk behavioral indicators, such as destructive operations or abusive API usage, allowing the model to prioritize features associated with severe malicious behavior.

By combining static structural analysis with heuristic behavioral profiling, MalScan AI achieves enhanced detection capability, capturing both conventional malicious patterns and complex behaviors that may evade purely lexical or rule-based detection methods.

Table 5. Summary of Feature Categories in MalScan AI

Category	Description	Example Indicators
Static Structural Features	Captures AST-based syntax patterns, function/class definitions, control flow, and complexity metrics.	Nested loops, conditional branching, function calls
Lexical Features	Extracts lexical elements such as imports, function calls, string usage, and dynamic execution.	<code>import os</code> , <code>eval()</code> , suspicious string literals
System API Abuse	Detects use of APIs that can modify system behavior or resources.	<code>ctypes</code> calls, <code>winreg</code> edits, <code>BlockInput</code>
Network Anomalies	Detects suspicious network communication or external control patterns.	<code>socket.bind</code> , <code>socket.listen</code> , Discord webhook URLs
Destructive Behavior	Identifies code that can delete files or damage the system.	Recursive <code>shutil.rmtree</code> with threading
Evasion Techniques	Detects anti-analysis, anti-debugging, or anti-virtualization methods.	<code>IsDebuggerPresent</code> , scanning for “VMware” strings
Obfuscation Indicators	Identifies hidden or encrypted code segments using entropy and encoding patterns.	High Shannon entropy, Base64-encoded strings

3.6 Model Training

Multiple machine learning classifiers are trained and evaluated to assess their suitability for malicious Python source code detection. The selected models represent diverse learning paradigms, including ensemble-based, linear, and kernel-based approaches, namely Random Forest, Gradient Boosting, Logistic Regression, and Support Vector Machine (SVM).

The dataset is partitioned into training and testing subsets using an 80:20 stratified split to preserve class distribution. Feature standardization using z-score normalization is applied only to models sensitive to feature magnitude,

specifically Logistic Regression and SVM, while tree-based models are trained using unscaled feature values. To ensure fair comparison, all classifiers are trained using the same feature set. Model selection is performed based on comparative evaluation using standard classification metrics, with k-fold cross-validation applied on the training data to improve reliability and reduce bias from a single data split.

3.7 Evaluation Metrics

The performance of the proposed detection system is assessed using standard classification metrics commonly employed in security-oriented machine learning tasks, including accuracy, precision, recall, and F1-score. Accuracy measures the overall proportion of correctly classified samples, while precision reflects the correctness of malicious predictions. Recall evaluates the system's ability to identify actual malicious samples, which is critical in malware detection due to the high risk associated with false negatives.

The F1-score, defined as the harmonic mean of precision and recall, is selected as the primary evaluation metric because it provides a balanced assessment under class imbalance conditions. To evaluate the robustness and stability of the trained models, k-fold cross-validation is employed, and the mean F1-score is used as the principal criterion for comparative analysis.

3.8 Experimental Setup

All experiments are conducted in a Python-based environment using the scikit-learn machine learning library. The experimental pipeline integrates feature extraction, preprocessing, model training, evaluation, and model persistence under consistent conditions to ensure fair comparison across classifiers.

The dataset is partitioned using an 80:20 stratified train-test split to maintain class balance, with k-fold cross-validation applied during training to assess model stability and generalization performance. Following comparative evaluation, the Random Forest classifier is selected for deployment due to its robustness, resistance to overfitting, and effectiveness in handling high-dimensional feature spaces.

The trained model is serialized using the joblib library and integrated into the MalScan AI web-based system. During runtime, the deployed model outputs a probability score representing the likelihood of malicious behavior, with a default decision threshold of 0.5 applied to classify source code samples as benign or suspicious.

3.9 Limitations

Despite its effectiveness, the proposed approach has several limitations. First, the dataset includes synthetically generated malicious samples to compensate for the limited availability of publicly labeled malicious Python source code. Although these samples improve dataset diversity, they may not fully capture the complexity and variability of real-world malware.

Second, the system relies primarily on static and heuristic analysis, limiting its ability to detect runtime-dependent behaviors, dynamic code loading, and environment-specific triggers. Finally, the evaluation was performed on a relatively small dataset compared to large-scale industrial malware detection systems. Future work will focus on expanding the dataset with additional real-world malware samples, exploring family-aware dataset partitioning strategies, and integrating dynamic analysis techniques to improve generalization performance.

4 Result and Discussion

This section presents the implementation and evaluation results of MalScan AI, a machine learning-based system for identifying malicious Python source code. The discussion focuses on model performance, system functionality, experimental results, and the effectiveness of the proposed static-analysis-based detection approach.

4.1 Model Performance

To determine the most appropriate model for identifying malicious code in source code, four machine learning models were tested: Random Forest, Gradient Boosting, Logistic Regression, and Support Vector Machine. The four models were evaluated using standard classification metrics, including accuracy, precision, recall, F1-score, and cross-validation score.

Dataset Preparation. The dataset used for model training and evaluation consisted of 1,066 Python source code files in total, out of which 534 were benign and 532 were malicious. This ensured a balanced class distribution for evaluation. The dataset was further divided into training and testing subsets using an 80:20 split, resulting in 852 training samples and 214 test samples for experimental evaluation using stratified sampling.

Performance Evaluation. Table 6 presents the comparative results of the four classifiers.

Table 6. Model Training and Evaluation Results

Model	Acc.	Prec.	Recall	F1	5-Fold CV
Random Forest	0.9720	0.9810	0.9626	0.9717	0.9488
Gradient Boosting	0.9673	0.9808	0.9533	0.9668	0.9460
Logistic Regression	0.9439	0.9524	0.9346	0.9434	0.9013
SVM	0.9206	0.9018	0.9439	0.9224	0.9131

CV refers to the mean 5-fold cross-validation score obtained during model evaluation. Within the collected dataset, the Random Forest classifier achieved the best performance, with an accuracy of 97.20%. This indicates that the model has a strong balance between detecting malicious source code accurately and reducing false positives.

Although the Gradient Boosting classifier performed competitively with an F1-score of 0.9668, it performed slightly below the Random Forest classifier. Logistic Regression and SVM produced lower accuracy values, indicating that they were less competent in identifying complex malicious patterns. Hence, the Random Forest classifier was chosen as the detection model for MalScan AI.

4.2 System Testing

System testing was performed to validate the correctness and dependability of the implemented MalScan AI system. Validation focused on core system functionalities, handling of invalid inputs, and correct system behavior when processing benign and malicious Python source code files.

Functional Testing. Table 7 summarizes the main test cases executed, including file upload validation, classification output, and scan history storage.

Table 7. Functional Test Cases and Results

Test Case ID	Test Description	Input	Expected Output	Actual Outcome	Status
T-01	Upload benign Python source code	Valid benign .py file	Classified as benign with low malicious probability	Classified as benign with low malicious probability	Pass
T-02	Upload malicious Python source code	Known malicious .py file	Classified as malicious with high confidence score	Classified as malicious with high confidence score	Pass
T-03	Upload unsupported file type	.txt/.exe file	Upload rejected with error message	Error message displayed	Pass
T-04	Multiple sequential uploads	Several .py files	Stable system response and correct classification	System stable	Pass
T-05	Scan history retrieval	Logged-in user	Previous scan records displayed	Scan history displayed	Pass

All functionality-related test cases (T-01 to T-05) were successful. This ensures that the system meets the basic functionality requirements. During testing, the system was evaluated against three primary qualities.

First, for detection accuracy, the model was able to distinguish between benign and malicious code. Benign Python scripts were processed without false alarm reports, while malicious scripts produced the expected high-level warnings from the Random Forest model.

Second, for input validation and error handling, the design efficiently validated unsupported file types such as `.txt` and `.exe`. Error messages were generated correctly to ensure that the backend engine was not subjected to inappropriate file types.

Third, for system stability and persistence, stress testing through a series of uploads revealed that the system was stable and did not crash or hang. Retrieval of scan history was successful, validating that the database integration was correct and that all data were persistently stored.

System Output and Visualization. To demonstrate the system’s capability in identifying malicious files, the analysis result screen is presented in Fig. 2.

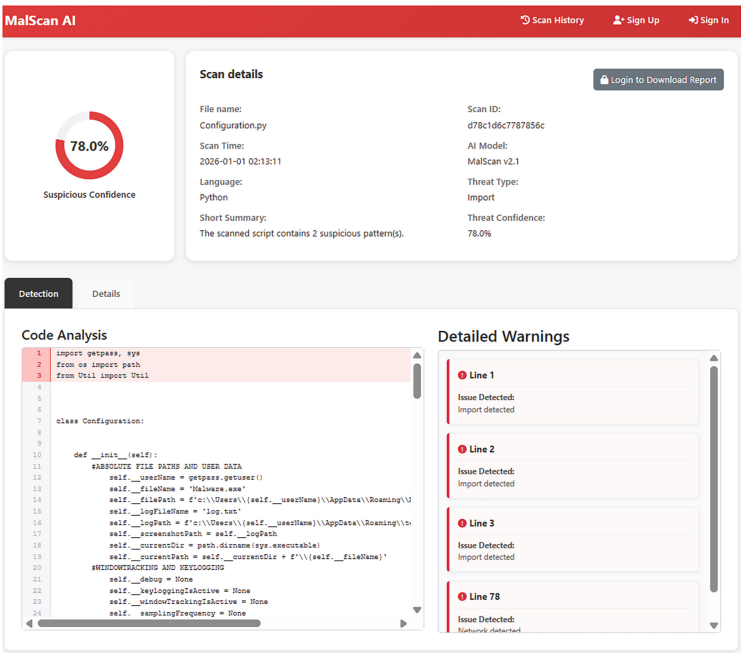


Fig. 2. MalScan AI report scan page.

■ MalScan AI

Python Malware Analysis Report

SUSPICIOUS

100.0%
Confidence

HIGH
Risk Level

■ Scan Information

File Name:	synthetic_suspicious_172.py
Scan Date & Time:	2025-12-28 09:59:43
Scan ID:	720ca3778df39d03
File Type:	Python Script (.py)
Scanner Version:	MalScan AI v2.1

■ Code Statistics

Lines of Code	82
Import Statements	8
Function Definitions	7
Cyclomatic Complexity	3
Character Entropy	4.24

■ Security Analysis

Issue #1: Suspicious imports detected

Description: Suspicious imports detected

Category: Import

Detected Value: 3

Issue #2: Network operations found

Description: Network operations found

Category: Network

Detected Value: 1

Fig. 3. MalScan AI download report example.

Figure 2 shows the MalScan AI report scan page, which contains file details and a suspicious confidence score in the upper section of the page. The lower section highlights specific lines of code that contain malicious behavior and warning boxes that explain the potential threat. Additionally, the system generates downloadable reports for record-keeping, as shown in Fig. 3.

The report includes information such as the scan time, result status, confidence level, and identified malicious indicators. The results obtained from evaluating MalScan AI demonstrate its usefulness as an early-stage detection tool.

The tool was able to produce appropriate classification results and feedback, helping users understand their detected security status.

5 Conclusion

This study presented MalScan AI, a web-based system for detecting malicious Python source code using static analysis and machine learning. The system allows users to upload Python (.py) files and classify them as benign or malicious without executing the source code, improving the safety of the analysis process. The implemented system integrates secure file upload, automated analysis, visualization and reporting, user management, feature extraction, model training, classification, and result reporting.

The evaluation results show that the Random Forest classifier achieved the best overall performance, with an accuracy of 97.20% and an F1-score of 0.9717. These results indicate that the proposed approach can provide accurate, interpretable, and non-intrusive early-stage malware detection for Python source code. By combining static source code review with machine learning, MalScan AI contributes to DevSecOps practices and supports security awareness among learners and early-stage developers.

Although the current implementation is limited by dataset size and its reliance on static analysis, the results demonstrate the feasibility of machine-learning-based malicious source code detection. Future work will focus on expanding the dataset with additional real-world malware samples, supporting multiple programming languages, and incorporating dynamic analysis capabilities.

Disclosure of Interests. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work.

References

1. Leka, C., Ntantogian, C., Karagiannis, S., Magkos, E., Verykios, V.S.: A comparative analysis of VirusTotal and desktop antivirus detection capabilities (2022). <https://doi.org/10.1109/IISA56318.2022.9904382>
2. VirusTotal: VirusTotal. <https://www.virustotal.com/>. Accessed Jan 2026
3. Hybrid Analysis: Free malware analysis service for dynamic and static analysis of files and URLs. <https://www.hybrid-analysis.com/>. Accessed Jan 2026
4. SonarSource: SonarQube. <https://www.sonarsource.com/>. Accessed Jan 2026
5. SiteGuarding: Malware scan – website malware removal and protection. <https://malware-scan.website-malware-removal.com/en.php>. Accessed Jan 2026
6. Tsfaty, C., Fire, M.: Malicious source code detection using transformer. arXiv preprint [arXiv:2209.07957](https://arxiv.org/abs/2209.07957) (2022). <https://arxiv.org/abs/2209.07957>
7. Yuk, C.K., Seo, C.J.: Static analysis and machine learning-based malware detection system using PE header feature values. *Int. J. Innov. Res. Sci. Stud.* **5**(4), 281–288 (2022)

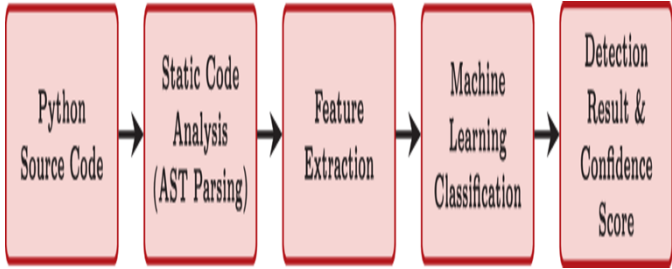
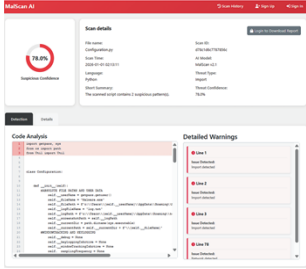
8. Sari, N.V., Aci, M.: Detection of malware by static analysis using machine learning methods. *Bilimsel Bilisim ve Teknoloji Dergisi (BIBTED)* **4**(2), 27–35 (2023). <https://doi.org/10.54047/bibtred.1309960>
9. Matsobane, N.O., Mokwena, S.: Malware detection using a random forest method trained on a balanced synthetic dataset. *Sci. Eng. Technol.* **5**(1), 203–212 (2025). <https://doi.org/10.54327/set2025/v5.i1.167>
10. Aslan, O., Samet, R.: A comprehensive review on malware detection approaches. *IEEE Access* **8**, 6249–6271 (2020). <https://doi.org/10.1109/ACCESS.2019.2963724>
11. Kumar, P.: Malicious code detection using machine learning. *Int. J. Res. Appl. Sci. Eng. Technol.* **12**(2), 298–304 (2024). <https://doi.org/10.22214/ijraset.2024.58295>
12. Farasat, T., Posegga, J.: Machine learning techniques for Python source code vulnerability detection, pp. 151–153 (2024). <https://doi.org/10.1145/3626232.3658637>
13. GeekComputers: Python. GitHub repository. <https://github.com/geekcomputers/Python>. Accessed 4 Jan 2026
14. ytisf: theZoo: Malware source code repository. GitHub repository (2025). <https://github.com/ytisf/theZoo/tree/master/malware/Source/Original>. Accessed Jan 2026
15. KrizzhSriskantharajah-UK: GitHub overview. <https://github.com/KrizzhSriskantharajah-UK>. Accessed Jan 2026

Author Queries

Chapter 8

Query Refs.	Details Required	Author's response
AQ1	Please check and confirm if the authors Given and Family names have been correctly identified.	

Alternative Texts for Your Images, Please Check and Correct them if Required

Page no	Fig/Photo	Thumbnail	Alt-text Description
7	Fig1	 <pre> graph LR A[Python Source Code] --> B[Static Code Analysis (AST Parsing)] B --> C[Feature Extraction] C --> D[Machine Learning Classification] D --> E[Detection Result & Confidence Score] </pre>	The image outlines a sequential process starting with Python source code, followed by static code analysis and abstract syntax tree parsing, then feature extraction, next machine learning classification, and finally detection result and confidence score. Each step leads to the next, indicating a workflow from code input to output evaluation.
14	Fig2		MalScan AI detected suspicious patterns in a Python script named Configuration.py with 78 percent confidence. The scan identified two suspicious patterns related to import statements on lines 1, 2, and 3, and a network issue on line 78. The code snippet shows import statements for getpass, sys, os, and Util modules, and part of a Configuration class defining file paths and user data variables.
17	Fig3		MalScan AI Python malware analysis report identifies the file synthetic_suspicious_172.py as suspicious with 100 percent confidence and a high risk level. The scan was performed on December 28, 2025, at 09:59:43 using MalScan AI version 2.1. The file is a Python script containing 82 lines of code, 8 import statements, 7 function definitions, a cyclomatic complexity of 3, and character entropy of 4.24. The security analysis lists two issues: suspicious imports detected with a detected value of 3, and network operations found with a detected value of 1.

